

docs.google.com – 全画面表示を終了するには、Esc を押します

TL1

01_09.Blenderの描画拡張

日本工学院専門学校
デザインカレッジ

今回の目標

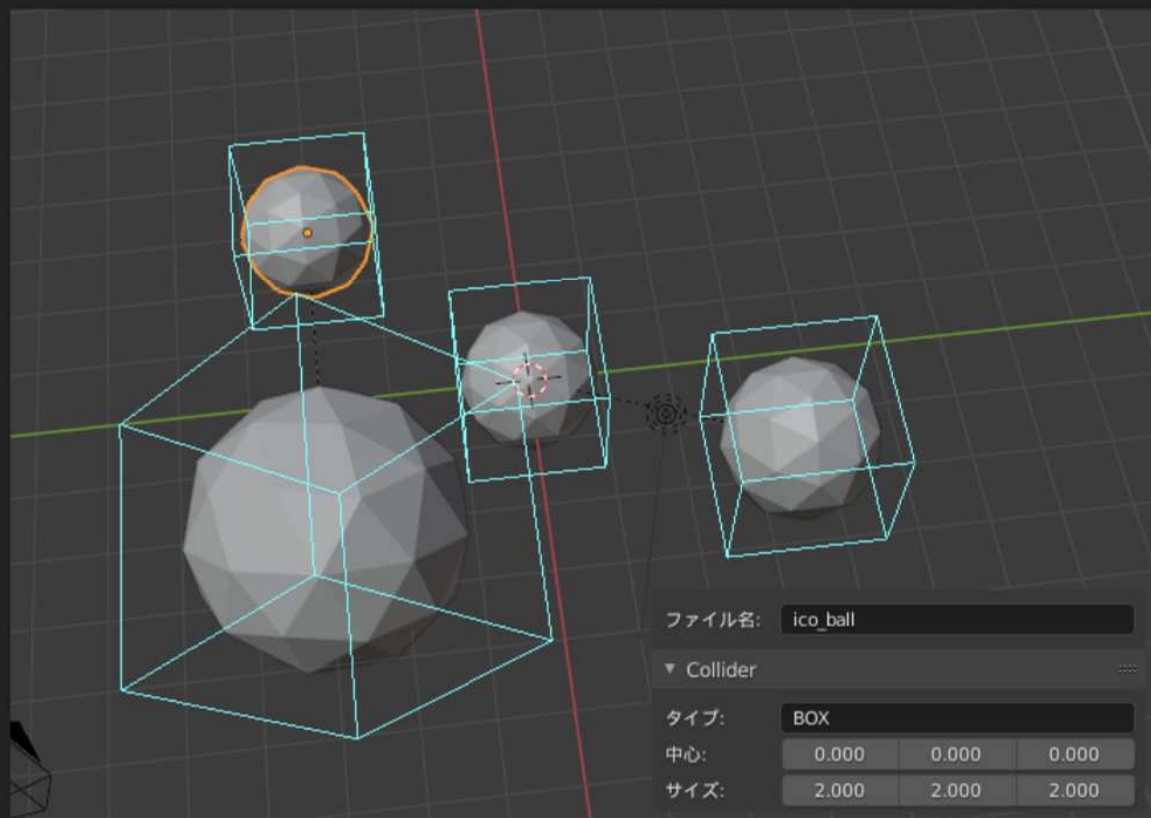
日本工学院専門学校
デザインカレッジ

- Blender Python API によって、3D描画を行う
- 各オブジェクトに追加で描画を行う

描画拡張

描画拡張

日本工学院専門学校
デザインカレッジ



BlenderではDirectXと同様に、描画命令を記述することでももとはなかった描画を追加することができる。

頂点を自分で指定したりもできるのでかなり自由自在な拡張が可能。

追加のパラメータを視覚化したりするのに役立つ。

差し当たりコライダーを追加したいが、見えないと数値のサイズ感などが分からないので、視覚化は重要。

モジュールのインポート

日本工学院専門学校
デザインカレッジ

```
import gpu
import gpu_extras.batch
```

スクリプトの先頭で、Blenderの描画関連モジュールをインポートする。

gpu は描画周りを手広くサポートするモジュール [リファレンス](#)

ビルトイン（あらかじめ組み込まれた）シェーダを使うこともできるし、
自分で頂点シェーダ、フラグメントシェーダ（ピクセルシェーダ）を書くこともできる。

gpu_extras.batch はジオメトリバッチを提供するモジュール

ジオメトリバッチは描画に使用する
「シェーダ、トポロジー、頂点バッファ、インデックスバッファ」を一つにまとめたもの。
公式ドキュメントにより使用が推奨されている。

コライダー描画クラス

日本工学院専門学校
デザインカレッジ

```
#コライダー描画
class DrawCollider:

    # 描画ハンドル
    handle = None

    #3Dビューに登録する描画関数
    def draw_collider():
        次ページにつづく
```

コライダー描画用のクラスを新たに定義する。
これはコライダー描画関数と、それに関連する静的メンバ変数をまとめる為だけのクラス。

メニューでもパネルでもオペレータでもないので、ブレンダーの組み込みクラスを継承しなくてよい。
また、「Blenderに登録するクラスリスト」に登録する必要もない。

描画関数

日本工学院専門学校
デザインカレッジ

```
class DrawCollider -draw_collider

#頂点データ
vertices = {"pos":[[0,0,0], [2,2,2]]}
#インデックスデータ
indices = [[0, 1]]

# ビルトインのシェーダを取得
shader = gpu.shader.from_builtin("UNIFORM_COLOR")

# バッチを作成(引数:シェーダ、トポロジー、頂点データ、インデックスデータ)
batch = gpu_extras.batch.batch_for_shader(shader, "LINES", vertices, indices = indices)

# シェーダのパラメータ設定
color = [0.5, 1.0, 1.0, 1.0]
shader.bind()
shader.uniform_float("color", color)
# 描画
batch.draw(shader)
```

処理の流れは次ページで解説。

解説

日本工学院専門学校
デザインカレッジ

- ① 後で指定する予定のシェーダーに合わせて頂点データとインデックスデータを用意する。
このシェーダ用の頂点データは座標のみ。XYZを2点分用意した。
ラインリスト用のインデックスデータとして、ラインの始点と終点で2個のインデックスを
ラインの本数分だけ2次元配列で用意する。今回はライン1本分用意。
- ② ユニフォーム（定数バッファ）の項目が色指定のみの3D描画用シェーダを取得。
この代わりに[参考](#)のようにすれば自分でGLSLのシェーダコードを書くこともできる。
GLSLはHLSLとほぼ同じ。
- ③ シェーダー、トポロジー、頂点データ、インデックスデータを指定してGPUバッチを作成する。
トポロジーは 'POINTS', 'LINES', 'TRIS', 'LINES_ADJ' から選択して指定。
- ④ シェーダーのバインドで、描画に使うシェーダーをセットする。
ユニフォーム（定数バッファ）の値をセットし、描画。

有効化時の処理

日本工学院専門学校
デザインカレッジ

level_editor.py

```
#Add-On有効化時コールバック
def register():
    # Blenderにクラスを登録
    for cls in classes:
        bpy.utils.register_class(cls)

    #メニューに項目を追加
    bpy.types.TOPBAR_MT_editor_menus.append(TOPBAR_MT_my_menu.submenu)
    #3Dビューに描画関数を追加
    DrawCollider.handle =
bpy.types.SpaceView3D.draw_handler_add(DrawCollider.draw_collider, (), "WINDOW", "POST_VIEW")
print("レベルエディタが有効化されました。")
```

アドオン有効化時に、3Dビューのクラスに描画関数を登録する。
登録時にハンドルが返ってくるので、変数に保存しておいて登録解除時に使う。

無効化時の処理

日本工学院専門学校
デザインカレッジ

level_editor.py

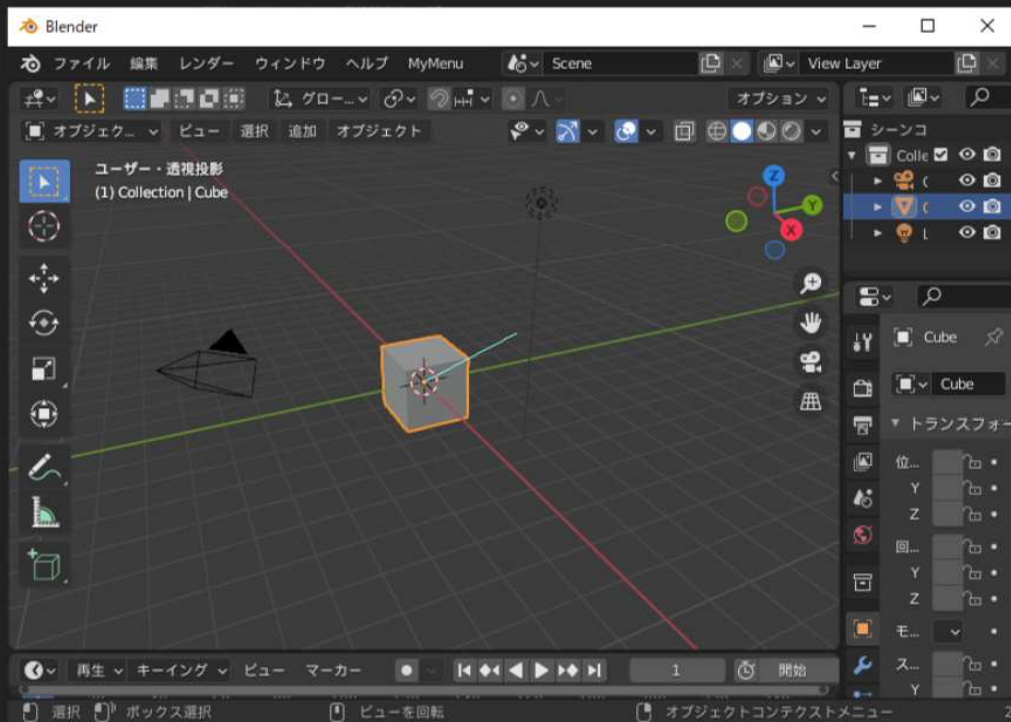
```
#Add-On無効化時コールバック
def unregister():
    #メニューから項目を削除
    bpy.types.TOPBAR_MT_editor_menus.remove(TOPBAR_MT_my_menu.submenu)
    #3Dビューから描画関数を削除
    bpy.types.SpaceView3D.draw_handler_remove(DrawCollider.handle, "WINDOW")

    # Blenderからクラスを削除
    for cls in classes:
        bpy.utils.unregister_class(cls)
    print("レベルエディタが無効化されました。")
```

アドオン無効化時に、3Dビューに登録した描画関数を登録解除する。

実行結果

日本工学院専門学校
デザインカレッジ



アドオンを有効化すると、左の図のような水色の線が追加されている。

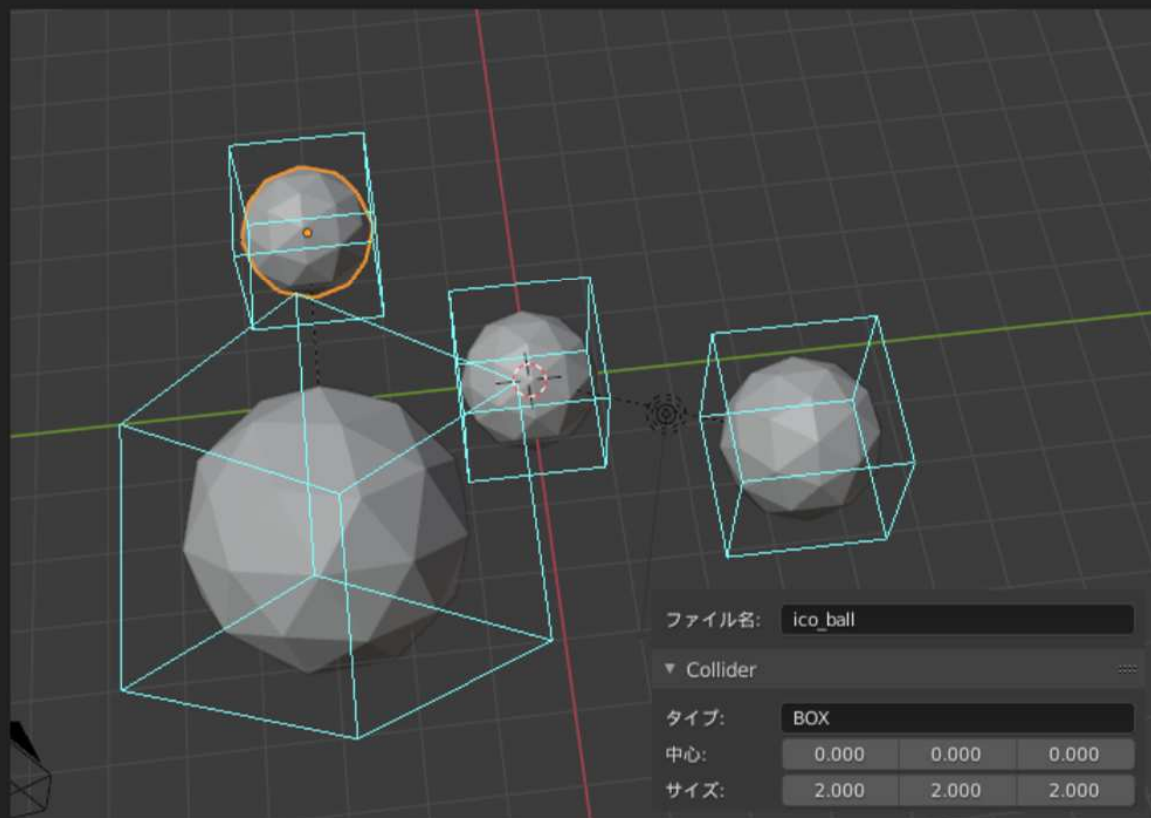
これが今回追加したシェードによる描画。

一見オブジェクトの中心から線が出ているように見えるが、単にワールドの原点から指定した座標通りに出ている。

オブジェクトに描画を追加

オブジェクトに描画を追加

日本工学院専門学校
デザインカレッジ



シーンにある各オブジェクトに
描画を付け足してみる。

今回は、一律同じ大きさのBoxを
全オブジェクトの周りに描画する。

モジュールのインポート

日本工学院専門学校
デザインカレッジ

```
import copy
```

スクリプトの先頭で、Python標準の copy モジュールをインポートしておく。

Pythonでは代入を行うとコピーではなく、参照が代入されるのが一般的な挙動。
リストを丸ごとコピーして増やしたい場合は、copyモジュールを使うと便利。

※数ページ後で使いたいので先に書いておいた

頂点データの変更

日本工学院専門学校
デザインカレッジ

```
class DrawCollider -draw_collider
```

変更

```
#頂点データ
vertices = {"pos":[]}
#インデックスデータ
indices = []
...省略...
```

全オブジェクトのBoxについて、

- 頂点データリスト（動的配列）
- インデックスデータリスト（動的配列）

を共有する方向で実装する。

先ほどは最初に座標とインデックスを直接指定していたが、後から各オブジェクト分のデータを追加していくので初期化時は空にしておく。

頂点オフセットの用意

日本工学院専門学校
デザインカレッジ

```
class DrawCollider -draw_collider
```

```
#各頂点の、オブジェクト中心からのオフセット
```

```
offsets = [
```

```
    [-0.5, -0.5, -0.5], #左下前
```

```
    [+0.5, -0.5, -0.5], #右下前
```

```
    [-0.5, +0.5, -0.5], #左上前
```

```
    [+0.5, +0.5, -0.5], #右上前
```

```
    [-0.5, -0.5, +0.5], #左下奥
```

```
    [+0.5, -0.5, +0.5], #右下奥
```

```
    [-0.5, +0.5, +0.5], #左上奥
```

```
    [+0.5, +0.5, +0.5], #右上奥
```

```
]
```

```
#立方体のX,Y,Z方向サイズ
```

```
size = [2, 2, 2]
```

続いて、立方体の頂点となる8点分のローカル座標をデータとして用意する。

オフセットと表現しているのは、これが最終的な座標でなく、各オブジェクトの中心からこの数値分ずらした位置が最終的な座標（ワールド座標）となるから。

ついでに立方体のサイズもここで指定しておく。今回は決め打ちで全方向の長さを2にする。単位は標準設定だとm（メートル）。

全オブジェクト走査

日本工学院専門学校
デザインカレッジ

```
class DrawCollider -draw_collider
```

```
#現在シーンのオブジェクトリストを走査
for object in bpy.context.scene.objects:
    #追加前の頂点数
    ① start = len(vertices["pos"])

    #Boxの8頂点分回す
    for offset in offsets:
        #オブジェクトの中心座標をコピー

    ② pos = copy.copy(object.location)
        #中心点を基準に各頂点ごとにずらす
        pos[0]+=offset[0]*size[0]
        pos[1]+=offset[1]*size[1]
        pos[2]+=offset[2]*size[2]
        #頂点データリストに座標を追加
        vertices['pos'].append(pos)
        つづく
```

シーンのオブジェクトリストをfor文で走査し、
全オブジェクトについて
オブジェクト1つずつ処理していく。

- ① 後で使うため、このオブジェクトの頂点データを追加する前に頂点データがいくつ登録済みか記録する
- ② 全オブジェクトで共有する頂点データリストに、このオブジェクトの8点分の座標を追加する

全オブジェクト走査

日本工学院専門学校
デザインカレッジ

つづき

```
#前面を構成する辺の頂点インデックス
indices.append([start+0,start+1])
indices.append([start+2,start+3])
indices.append([start+0,start+2])
indices.append([start+1,start+3])
#奥面を構成する辺の頂点インデックス
indices.append([start+4,start+5])
indices.append([start+6,start+7])
indices.append([start+4,start+6])
indices.append([start+5,start+7])
#手前と奥を繋ぐ辺の頂点インデックス
indices.append([start+0,start+4])
indices.append([start+1,start+5])
indices.append([start+2,start+6])
indices.append([start+3,start+7])
```

for文内のつづき。

ラインリストなので、Boxを構成する
全ての辺について12本分のインデックスデータを
追加する。

この時、前ページの処理で記録しておいた
start を活用する。

オブジェクト1つ分の開始頂点番号を基準にして
その何番後ろかを+で指定することで、
for文のどの周回でも通用するようにしている。

終わり

日本工学院専門学校
デザインカレッジ

参考用

```
# ビルトインのシェーダを取得
shader = gpu.shader.from_builtin("UNIFORM_COLOR")

# バッチを作成(引数: シェーダ、トポロジー、頂点データ、インデックスデータ)
batch = gpu_extras.batch.batch_for_shader(shader, "LINES", vertices, indices = indices)

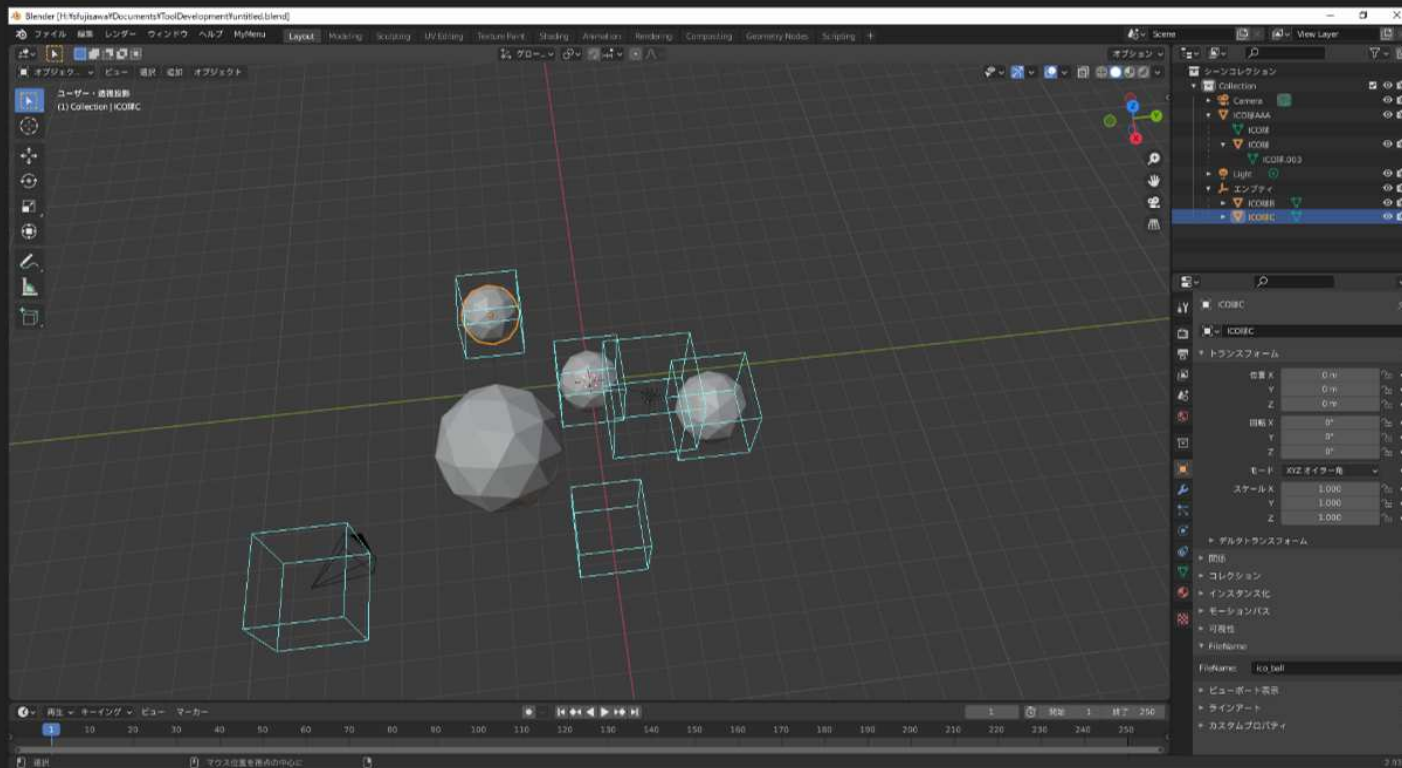
# シェーダのパラメータ設定
color = [0.5, 1.0, 1.0, 1.0]
shader.bind()
shader.uniform_float("color", color)
# 描画
batch.draw(shader)
```

先ほどまでで変更・追加は完了。この後、既に行った上記の処理が続くことで、今回追加した頂点データ、インデックスデータを反映した描画が行われる。

全オブジェクト分を1セットの頂点データリストとインデックスデータリストにまとめたので、draw は1回でよい。

実行結果

日本工学院専門学校
デザインカレッジ



シーン上の全オブジェクトに
一律で同じサイズ、
同じ方向を向いたボックスが
追加描画されるようになった。

まとめ

日本工学院専門学校
デザインカレッジ

今回ははじめての描画拡張ということで、最小限の描画から、オブジェクトごとの追加描画までを行った。

次回は実用を兼ねた拡張として
今回のBoxをBoxコライダーに改造してみたい。

TL1

01_09.Blenderの描画拡張

日本工学院専門学校
デザインカレッジ